SML v2

SML v2 trains and runs language models with MLX on Apple Silicon. Run commands from the repository root with Python 3.12.13 through `uv run`. Creating bundles and training or resuming runs require local APFS storage; reading a run's latest checkpoint requires a local filesystem.

Model and changes from v1

The model is a decoder-only Transformer with RMSNorm, grouped-query attention, SwiGLU, rotary position embeddings, and tied input/output embeddings. The SentencePiece BPE tokenizer uses byte fallback and unknown/BOS/EOS/padding IDs `0/1/2/3`.

Default	v1	v2
Runtime	PyTorch	MLX
Parameters	About 48.8 million	About 99.9 million
Vocabulary / hidden width	24,576 / 512	28,672 / 768
Layers / query heads / KV heads	12 / 8 / 2	12 / 12 / 3
Feed-forward width	1,536	2,176
Pretraining / inference context	1,024 / 2,048 tokens	1,024 / 8,192 tokens
SWAG objective	Predict the correct ending's tokens	Rank all four endings

V2 prepares reusable datasets before training and saves models with their tokenizer and resumable training state. Base training uses BF16 working weights, FP32 master weights and optimizer state, and compiled kernels by default. See the model, tokenizer, and optimizer configurations for detailed defaults.

V1 checkpoints and standalone tokenizer files are incompatible with v2; there is no conversion command. V2 provides generation and HellaSwag/Winogrande evaluation, but does not expose v1's HTTP server or HumanEval script.

Command line

```
uv run python -m sml --help
```

Each command accepts `--help` and `--config path/to/config.toml`. Settings resolve as defaults → TOML → explicit CLI flags. A TOML file must contain only the selected command's table and its subtables, such as `[train]` or `[prepare.pretraining]`. Unknown fields and duplicate semantic fields are rejected. See the CLI schemas for settings available through TOML.

Tokenizer and pretraining data

Supply a directory of compressed `*.jsonl.zst` files with a `text` field in each JSON row, then build the tokenizer and prepared data:

```
mkdir -p v2/output
uv run python -m sml tokenize \
  --input data/corpus --output v2/output/tokenizer
uv run python -m sml prepare pretraining \
  --input data/corpus --tokenizer v2/output/tokenizer \
  --output v2/output/pretraining-data
```

Both commands discover nonhidden matching files directly inside the input directory. By default, each scans up to **100 files** and the first **8,192 physical lines per file**, with seeded file selection and ordering. Text is normalized by replacing null characters and collapsing whitespace, then filtered to **100–16,384 UTF-8 bytes** per document.

Tokenizer training takes a seeded sample of at most **100,000 unique normalized documents / 128 MiB** from all scanned prefixes. Whole documents are retained, so the byte budget may not fill exactly. Adjust sample size and scan limits with:

```
uv run python -m sml tokenize \
  --input data/corpus --output v2/output/tokenizer-small \
  --max-documents 50000 --max-corpus-bytes 67108864 \
  --max-files 40 --max-rows-per-file 8192 --sampling-seed 42
```

Pretraining has independent scan limits and does not inherit tokenizer sample caps or deduplication. To prepare more text, save this as `pretraining.toml` and run `uv run python -m sml prepare pretraining --config pretraining.toml`, using a new output directory:

```
[prepare.pretraining]
input = "data/corpus"
tokenizer = "v2/output/tokenizer"
output = "v2/output/pretraining-data-large"

[prepare.pretraining.corpus]
max_files = 300
max_rows_per_file = 32768
```

Prepared data contains fixed-width token rows, deterministically shuffled within bounded windows, in memory-mapped NPY shards plus a copied tokenizer. Packing drops the final incomplete row and uses no padding; full verification and training reject padding tokens in prepared rows.

Base training

```
uv run python -m sml train \
  --data v2/output/pretraining-data --output v2/output/base-run
```

Training defaults to microbatch size **1**, **8** gradient accumulation steps, and **one epoch with no step limit**. An incomplete microbatch is dropped at an epoch boundary; an incomplete accumulation window still produces

an update. Progress reports loss, learning rate, and processed rows to stderr.

A fresh run's learning-rate schedule follows its planned update budget, with 1% warmup and cosine decay. Set `schedule_steps` and `warmup_steps` under `[train.optimizer]` to override it. The resolved schedule is saved and preserved on resume, even when extending the run.

Step and epoch limits are absolute totals; training stops when either is reached. For 100,000 updates, set `--maximum-steps 100000` **and** an epoch limit large enough to supply them. The final checkpoint is the selected model; intermediate checkpoints support recovery, with no validation-based selection or early stopping. Nonfinite updates stop training before commit, preserving the last published checkpoint.

Resume with operational overrides only:

```
uv run python -m sml train --resume v2/output/base-run \
  --maximum-epochs 2 --checkpoint-interval 100
```

Allowed overrides are `maximum_steps`, `maximum_epochs`, `log_interval`, and `checkpoint_interval`. Use `--data` to relocate a prepared bundle only if its identity matches. Model, optimizer, precision, loader, seed, compile, and output settings cannot change. To continue a completed run, raise every reached stopping limit beyond the saved progress. These rules also apply to `finetune`.

SWAG LoRA fine-tuning and export

Prepare SWAG from the base checkpoint, fine-tune, then export merged weights:

```
uv run python -m sml prepare swag \
  --checkpoint v2/output/base-run --revision main --output v2/output/swag-data
uv run python -m sml finetune \
  --checkpoint v2/output/base-run --data v2/output/swag-data \
  --output v2/output/swag-run
uv run python -m sml export \
  --checkpoint v2/output/swag-run --output v2/output/swag-export
```

Preparation resolves `main` to a Hugging Face commit SHA; use that SHA to reproduce the dataset revision. It writes reusable candidate arrays in length buckets from 64 to 8,192 tokens. Repeating preparation with the same configuration, checkpoint, and output reuses a fully verified bundle without contacting the dataset provider. Prepare and fine-tune against the **same base checkpoint**: advancing the base run or changing preparation settings requires a new data output directory.

SWAG uses cross-entropy over four candidate scores, each the mean continuation token log-likelihood including EOS. Defaults are microbatch **1**, accumulation **8**, and stopping at **8,192 updates or 5 epochs**, whichever comes first. Its default learning-rate schedule is 8,192 updates with 81 warmup updates. LoRA defaults are rank `16`, alpha `32`, dropout `0.05`, Q/K/V/O targets, and rank-stabilized scaling (`alpha / sqrt(rank)`).

Preparation and fresh fine-tuning default to **8× YaRN scaling**, extending the original 1,024-token context to 8,192. Set `--rope-scaling-factor` on both commands for another capacity. Lowering the factor also requires lowering preparation's `--maximum-length` from 8,192; populated buckets must fit the fine-tuning context. The saved scale is preserved on resume and export.

```
uv run python -m sml finetune --resume v2/output/swag-run \
  --maximum-steps 16000 --maximum-epochs 6
```

A LoRA run contains its frozen base snapshot, tokenizer, and adapter checkpoints. Export fully verifies the latest checkpoint and merges the adapters into a portable model/tokenizer bundle without training state.

Inference and evaluation

```
uv run python -m sml infer --checkpoint v2/output/base-run \
  --max-new-tokens 128 "Once upon a time"
uv run python -m sml evaluate --checkpoint v2/output/base-run \
  --task hellaswag --task winogrande --output v2/output/evaluation.json
```

Both commands accept a pretraining run, LoRA run, or merged export. Generation defaults to greedy decoding; set `--temperature`, `--top-p`, and `--seed` for seeded sampling. Add `--include-prompt` to render the prompt with the response. Use `--limit` with evaluation for a smoke run.

Base-run inference and evaluation default to an **8,192-token context** with 8× YaRN scaling; LoRA runs and exports use their saved scale. Override with `--rope-scaling-factor` (at least 1), or `rope_scaling_factor` under `[infer.runtime]` / `[evaluate.runtime]`. A factor of 1 restores the original context length. Inference requires `use_cache = true`.

Evaluation writes an immutable JSON result containing provider metrics and model, tokenizer, task, dataset, context, and ordered-request provenance. Use a new output path for changed results; an otherwise identical result differing only in the provider's execution date reuses the original file.

Artifacts and verification

Pass complete artifact directories to commands. Runs resolve through their recovered `latest.json`; individual checkpoint directories are not model inputs. Inference, evaluation, and SWAG preparation accept base runs, LoRA runs, or exports. New fine-tuning requires a pretraining run, and export requires a LoRA run.

Artifact	Contents
Tokenizer	`manifest.json`, `tokenizer.model`, `tokenizer.vocab`
Pretraining data	`manifest.json`, copied `tokenizer/`, NPY `shards/`
SWAG data	`manifest.json`, bucketed input IDs, labels, and masks
Base run	`run.json`, `latest.json`, `tokenizer/`, model and training checkpoints
LoRA run	Run metadata, `tokenizer/`, frozen `base/`, adapter and training checkpoints
Merged export	`manifest.json`, `model.safetensors`, `tokenizer/`

Runs retain one latest checkpoint in steady state. Writable resume recovers and prunes crash leftovers after full verification. Read-only workflows, including export, can recover a stale latest index in memory without

modifying the source.

```
uv run python -m sml verify v2/output/base-run
uv run python -m sml verify --full v2/output/swag-export
```

Default `manifest-trusted` verification checks manifests, identities, paths, structure, and payload sizes without rehashing all payload contents. `--full` adds payload hash verification and semantic checks. Inference and evaluation also accept `--full`; training, resume, SWAG preparation, and export always use full verification.

See the benchmark guide for reproducible performance and Metal memory measurements.